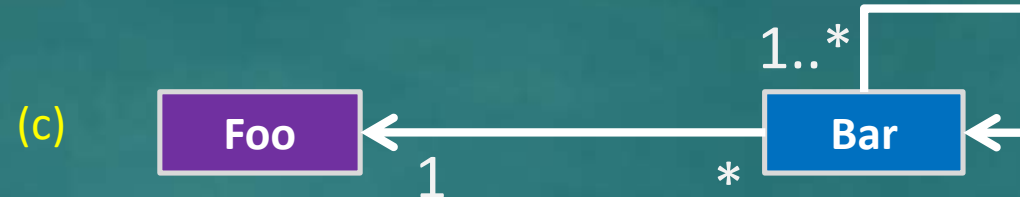
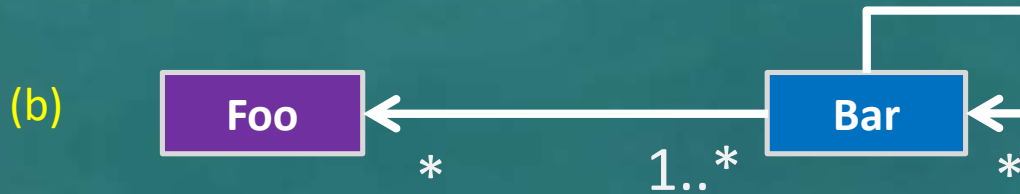
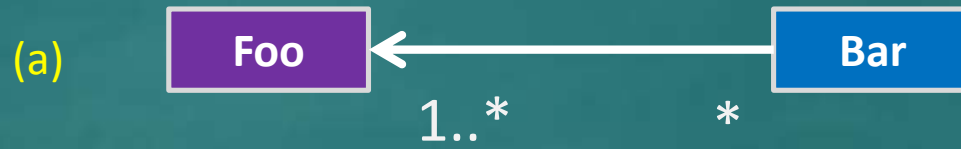


WARNING

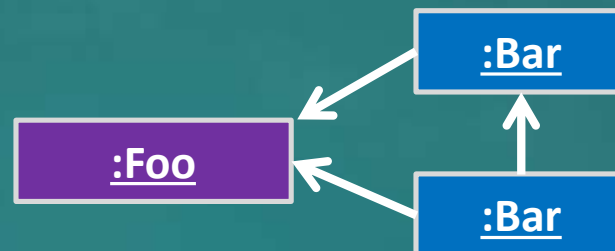
These slides are **not optimized for printing** or exam preparation. These are for lecture delivery only.

These slides contain a lot of animations. For optimal results, **watch in slideshow mode**.

Which class diagram(s) match the object diagram?



Class diagram → class structure
implemented in the code



Object diagram → object structure
at run-time

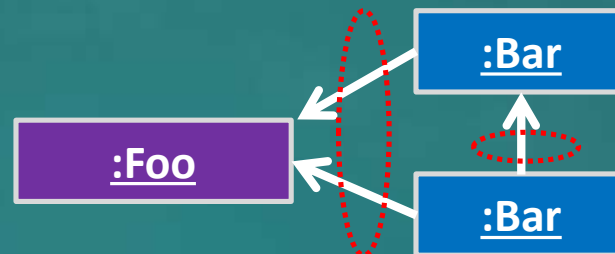
Learning outcome → able to verify they match

Answer this in the Canvas quiz

Q₁

1. The class diagram matches the given object diagram

- ☐ True
- ☐ False

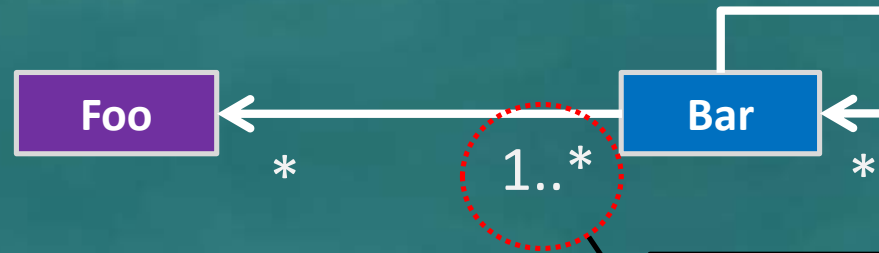


Approach: check different aspects (classes, associations, navigabilities, multiplicities, ...) until you find a mismatch.

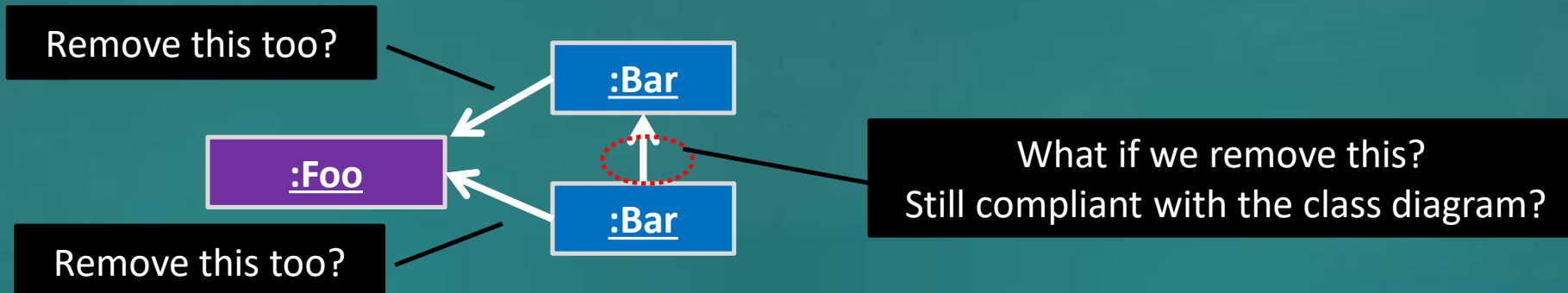
Q₂

2. The class diagram matches the given object diagram

- True
- False



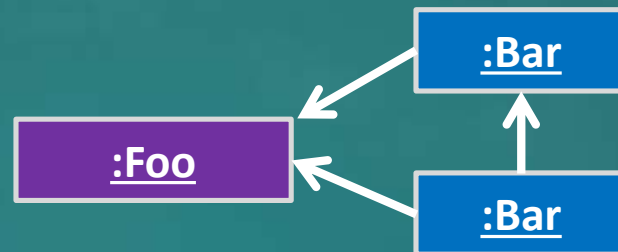
This says a **Bar** should be connected to at least one **Foo**. Agree?



Q₃

3. The class diagram matches the given object diagram

- ☐ True
- ☐ False



Tough week in the iP?

- **Good news:** *Technical difficulties* wise, last week can be considered the high bar for the semester.
- **If you survived it,** you can survive the rest (unless you make things harder by yourself).
- **If you are struggling,** there is still time to catch up.

One safety-net (i.e., 1-to-1 debugging help from tutors) removed.

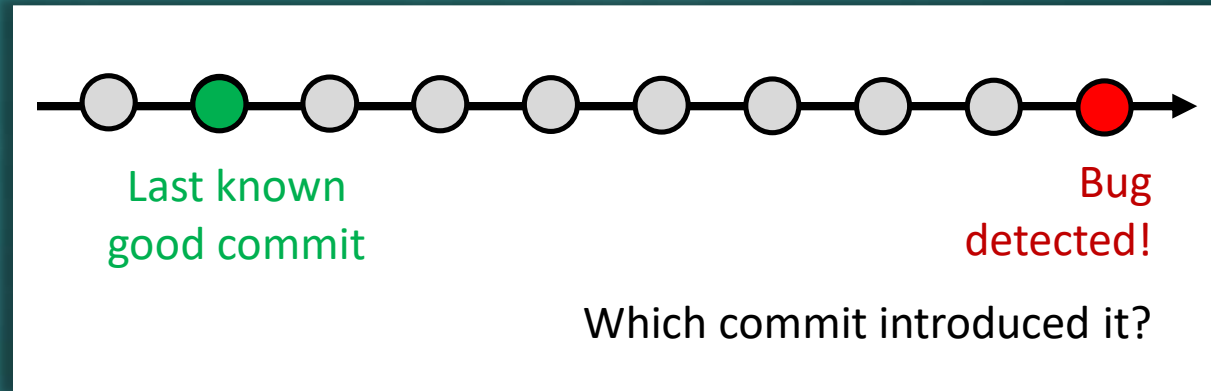
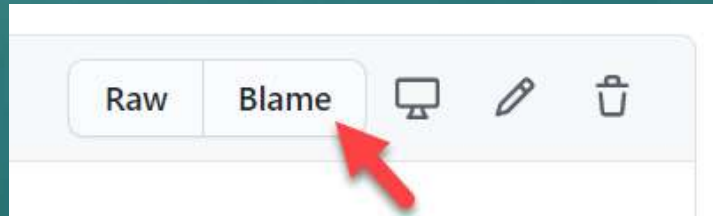
Another (i.e., forum) is provided, but do **keep increasing your self-reliance**.

If you posted an issue and it got resolved. What next?

1. Learn from it. 2. Help others with similar issues.

Q

Git and GitHub has a feature called **blame**.
What does it do?



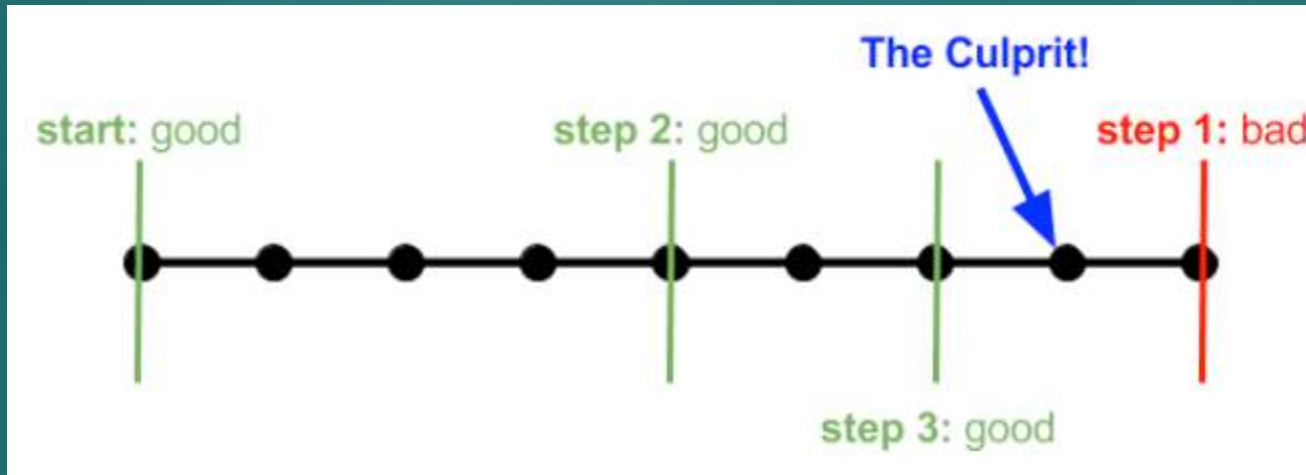
- Tells you who edited each line
- Helps you do a binary search of the commits to locate the commit that introduced a certain behavior to the code base

Why?

This Git feature is called **bisect**

e.g., to learn what lapses caused it to happen
to find author/reviewers accountable
to decide who should fix it

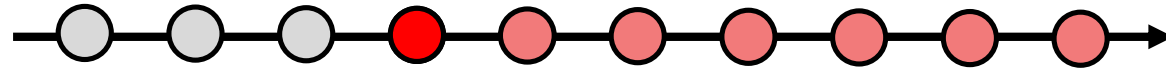
Git Bisect



Use the command `git bisect start` to begin the process.

Use `git bisect good` if the bug is not present in this commit, or `git bisect bad` if it is.

Use `git bisect reset` to exit the bisect mode and return to your original branch



You introduced it!

Bug
detected!

But... but...
I scored good
grades for all my
courses!



You need to be **able to do** stuff!

Focus on *learning*!

Score more with less work? Not necessarily.
Learn more with less work? **YES!**

Week 5

Topics:

- [W5.1] Requirements: Intro ⚡⚡
- [W5.2] Requirements: Gathering ⚡⚡⚡
- [W5.3] Requirements: Specifying ⚡
- [W5.4] Code Quality ⚡
- [W5.5] Refactoring ⚡⚡
- [W5.6] Assertions 🎓
- [W5.7] Java: streams 🎓
- [W5.8] RCS: Managing Pull Requests II ⚡⚡

Admin:

1. Submit weekly quiz **P**
2. [optional] Submit mid-term feedback for the course ⌚ Sat, Feb 17th 2359
3. Prelim peer evaluation on TEAMMATES ⌚ Thu, Feb 15th 2359 **P**

iP:

1. Generate a new JAR file
2. Write some full commit messages
3. Add Increments as PRs: A-Assertions, A-CodeQuality, A-Streams
4. Add Increment: A-CI
5. Add an extension

tP:

1. 👥 Brainstorm user stories ⌚ before the tutorial
2. 👥 Choose user stories for v1.2 ⌚ before/during the tutorial







1 billion downloads





FAIL



FAIL



WIN



FAIL



WIN

Requirements

Not easy!



ESTABLISHING requirements

SURVEYS

INTERVIEWS

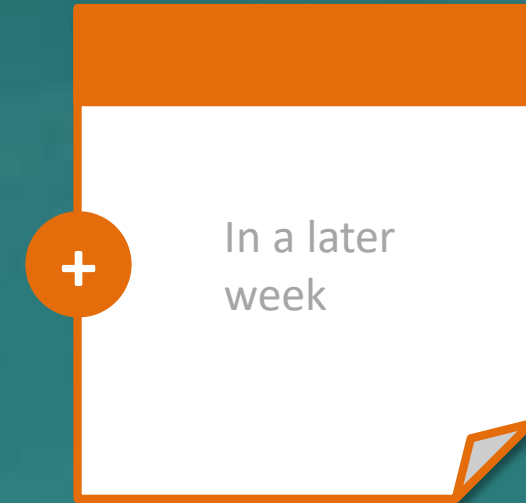
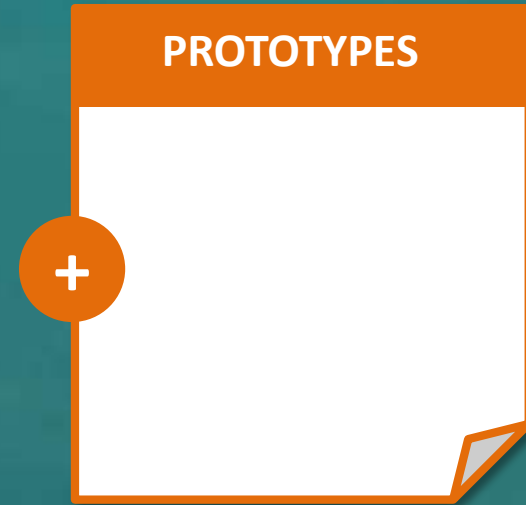
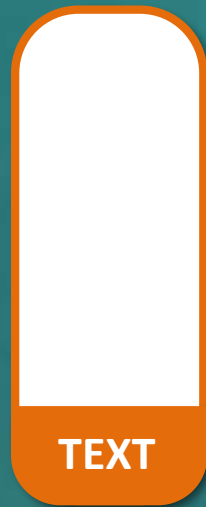
FOCUS GROUPS

PROD. STUDY

OBSERVE

BRAINSTORM

SPECIFYING requirements



Week 5

Topics:

- [W5.1] Requirements: Intro ⚡⚡
- [W5.2] Requirements: Gathering ⚡⚡⚡
- [W5.3] Requirements: Specifying ⚡
- [W5.4] Code Quality ⚡
- [W5.5] Refactoring ⚡⚡
- [W5.6] Assertions 🎓
- [W5.7] Java: streams 🎓
- [W5.8] RCS: Managing Pull Requests II ⚡⚡

Admin:

1. Submit weekly quiz P
2. [optional] Submit mid-term feedback for the course 🕒 Sat, Feb 17th 2359
3. Prelim peer evaluation on TEAMMATES 🕒 Thu, Feb 15th 2359 P

iP:

1. Generate a new J
2. Write some full c
3. Add Increments
4. Add Increment: `CodeQuality`, A-S
5. Add an extension

There are two ways to do this one:

😊 The expected way

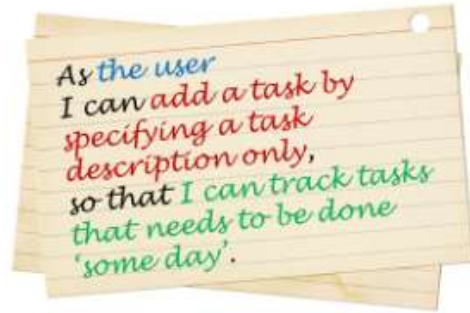
😞 The short way

tP:

1. 👤 Brainstorm user stories 🕒 before the tutorial
2. 👤 Choose user stories for v1.2 🕒 before/during the tutorial

tP: Gather requirements

1 👤 Brainstorm user stories ⌚ before the tutorial



- Learn [the recipe for brainstorming user stories](#) in the textbook → Specifying Requirements → UserStories → Usage.

Also learn about brainstorming and user stories.

We recommend you view the following video, as it has some additional commentary on the recipe to be used:

-  **Video** Additional commentary on the recipe to be used

- **Follow the steps in the recipe mentioned above to arrive at user stories for the product**, with your team members. If you didn't attend the lecture, please watch the lecture recording to learn more about this step.
Not following the recipe mentioned above could result in a different set of user stories than otherwise.
- **User stories for what version?** At this stage, collect user stories to cover at least the final version you hope to deliver at the end of the semester. It is OK to go even beyond that (reason: we are simulating a project that will continue even after the semester is over).
Do not omit user stories already covered by the features in AB3 i.e., the user story should be recorded even



tP tasks for this week

- ☐ Define a *persona*
- ☐ Define the problem *scope*
- ☐ Write a *narrative*
- ☐ Extract *user stories*
- ☐ Select *smallest set* of user stories
- ☐ Define features
- ☐ Define UI commands

Week 5

Topics:

- [W5.1] Requirements: Intro ⚡⚡
- [W5.2] Requirements: Gathering ⚡⚡⚡
- [W5.3] Requirements: Specifying ⚡
- [W5.4] Code Quality ⚡
- [W5.5] Refactoring ⚡⚡
- [W5.6] Assertions 🎓
- [W5.7] Java: streams 🎓
- [W5.8] RCS: Managing Pull Requests II ⚡⚡

Admin:

1. Submit weekly quiz **P**
2. [optional] Submit mid-term feedback for the course **🕒 Sat, Feb 17th 2359**
3. Prelim peer evaluation on TEAMMATES **🕒 Thu, Feb 15th 2359** **P**

iP:

1. Generate a new JAR file
2. Write some full commit messages
3. Add Increments as PRs: **A-Assertions**, **A-CodeQuality**, **A-Streams**
4. Add Increment: **A-CI**
5. Add an extension

tP:

1. 👤 Brainstorm user stories **🕒 before the tutorial**
2. 👤 Choose user stories for v1.2 **🕒 before/during the tutorial**

Week 5

Topics:

- [W5.1] Requirements: Intro ⚡⚡
- [W5.2] Requirements: Gathering ⚡⚡⚡
- [W5.3] Requirements: Specifying ⚡
- [W5.4] Code Quality ⚡
- [W5.5] Refactoring ⚡⚡
- [W5.6] Assertions 🎓
- [W5.7] Java: streams 🎓
- [W5.8] RCS: Managing Pull Requests II ⚡⚡

Admin:

1. Submit weekly quiz [P](#)
2. [optional] Submit mid-term feedback for the course ⌚ Sat, Feb 17th 2359
3. Prelim peer evaluation on TEAMMATES ⌚ Thu, Feb 15th 2359 [P](#)

iP:

1. Generate a new JAR file
2. Write some full commit messages
3. Add Increments as PRs: A-Assertions, A-CodeQuality, A-Streams
4. Add Increment: A-CI
5. Add an extension

tP:

1. 👤 Brainstorm user stories ⌚ before the tutorial
2. 👤 Choose user stories for v1.2 ⌚ before/during the tutorial

Week 5

Topics:

- [W5.1] Requirements: Intro ⚡⚡
- [W5.2] Requirements: Gathering ⚡⚡⚡
- [W5.3] Requirements: Specifying ⚡
- [W5.4] Code Quality ⚡
- [W5.5] Refactoring ⚡⚡
- [W5.6] Assertions 🎓
- [W5.7] Java: streams 🎓
- [W5.8] RCS: Managing Pull Requests II ⚡⚡

Admin:

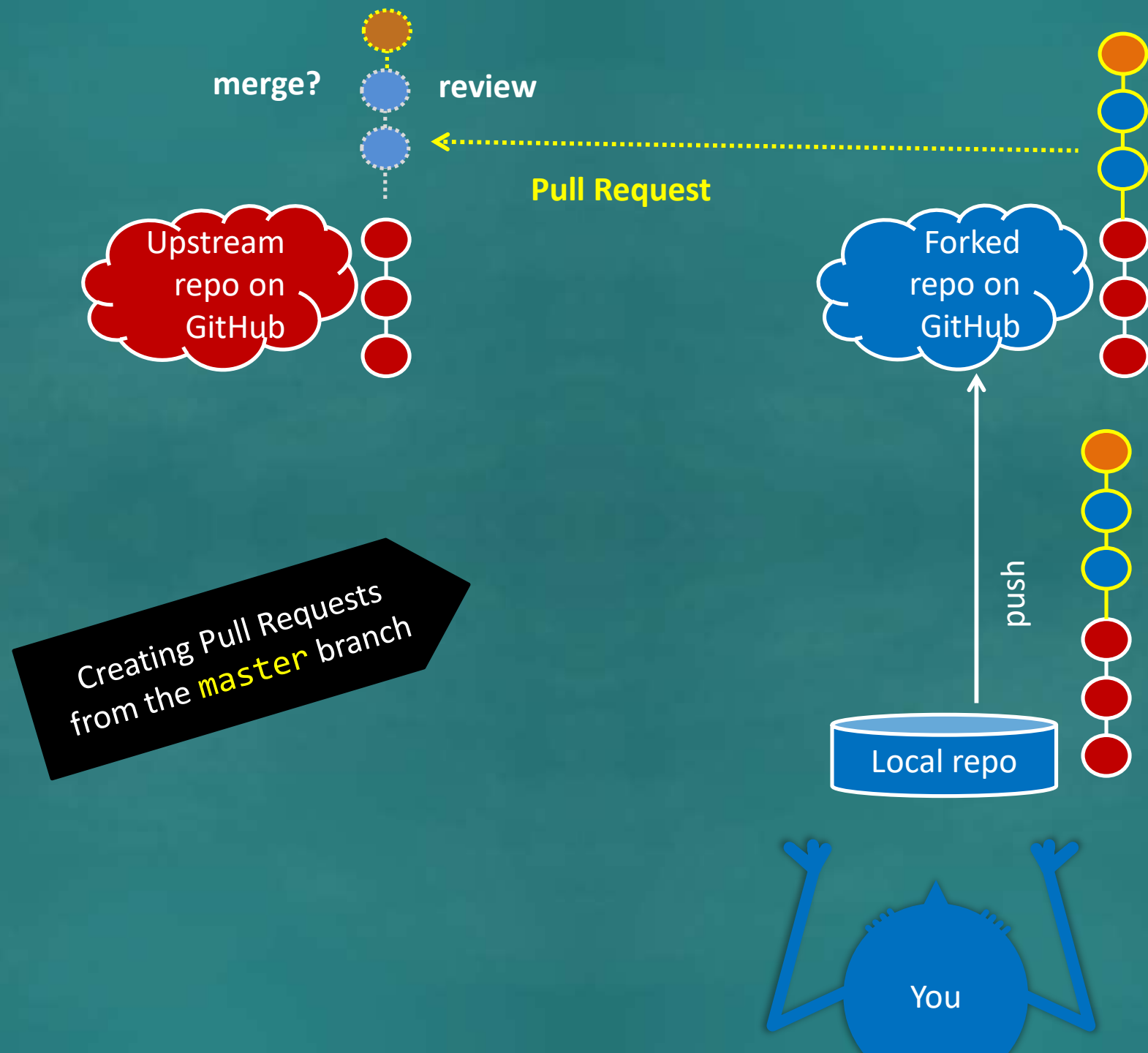
1. Submit weekly quiz P
2. [optional] Submit mid-term feedback for the course 🕒 Sat, Feb 17th 2359
3. Prelim peer evaluation on TEAMMATES 🕒 Thu, Feb 15th 2359 P

iP:

1. Generate a new JAR file
2. Write some full commit messages
3. Add Increments as PRs: A-Assertions, A-CodeQuality, A-Streams
4. Add Increment: A-CI
5. Add an extension

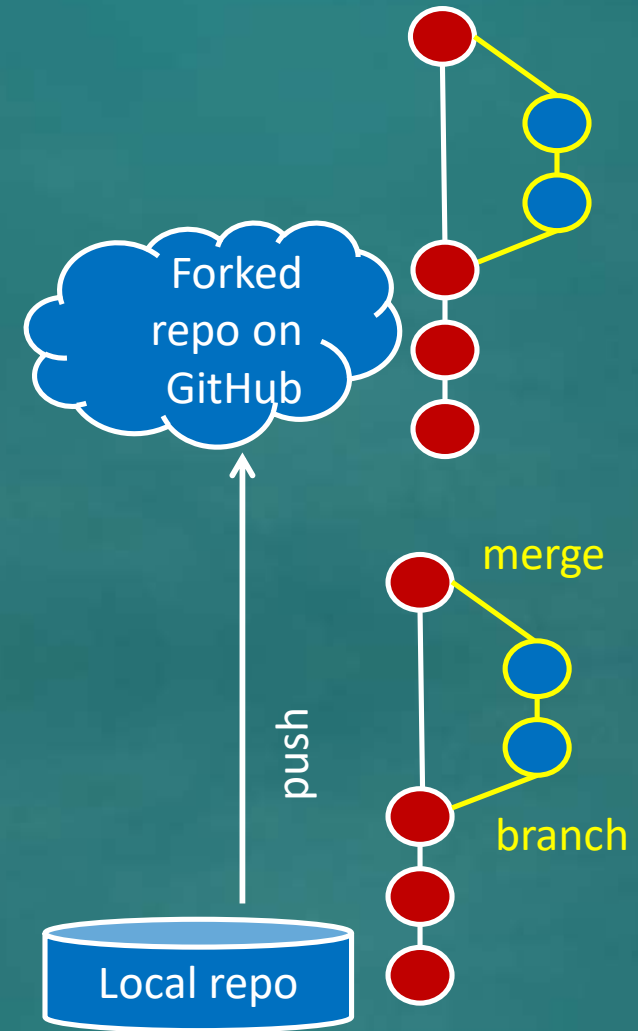
tP:

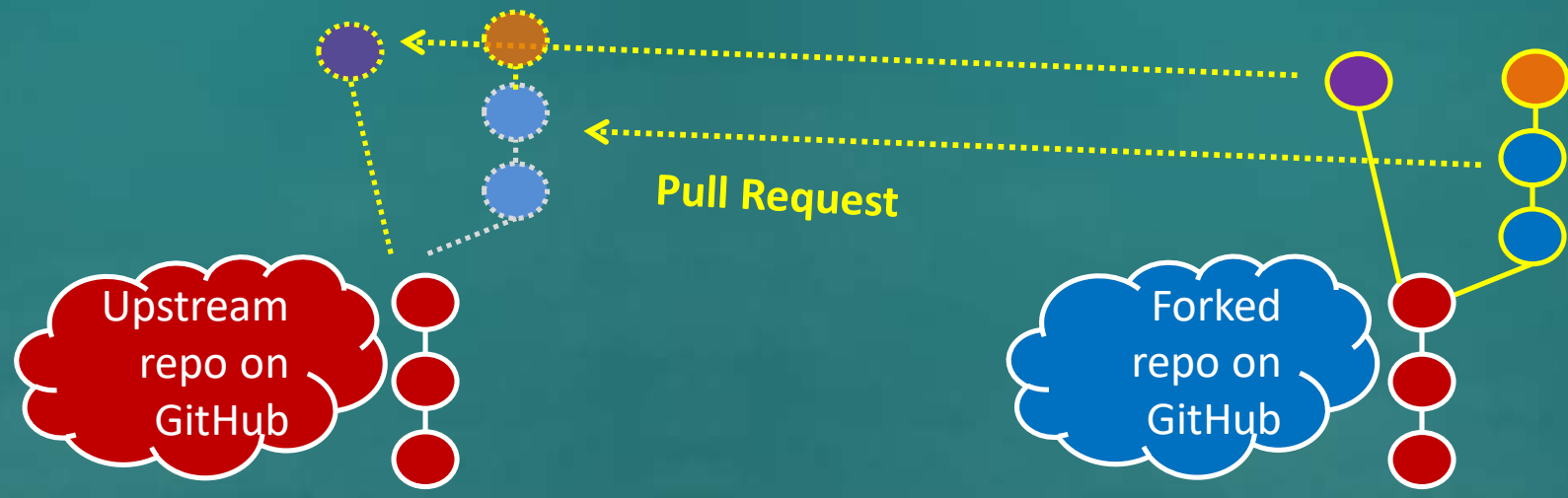
1. 👤 Brainstorm user stories 🕒 before the tutorial
2. 👤 Choose user stories for v1.2 🕒 before/during the tutorial



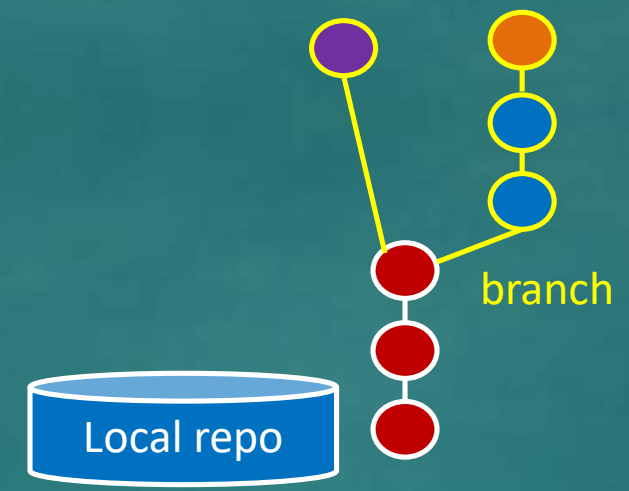


Using branches,
merging locally





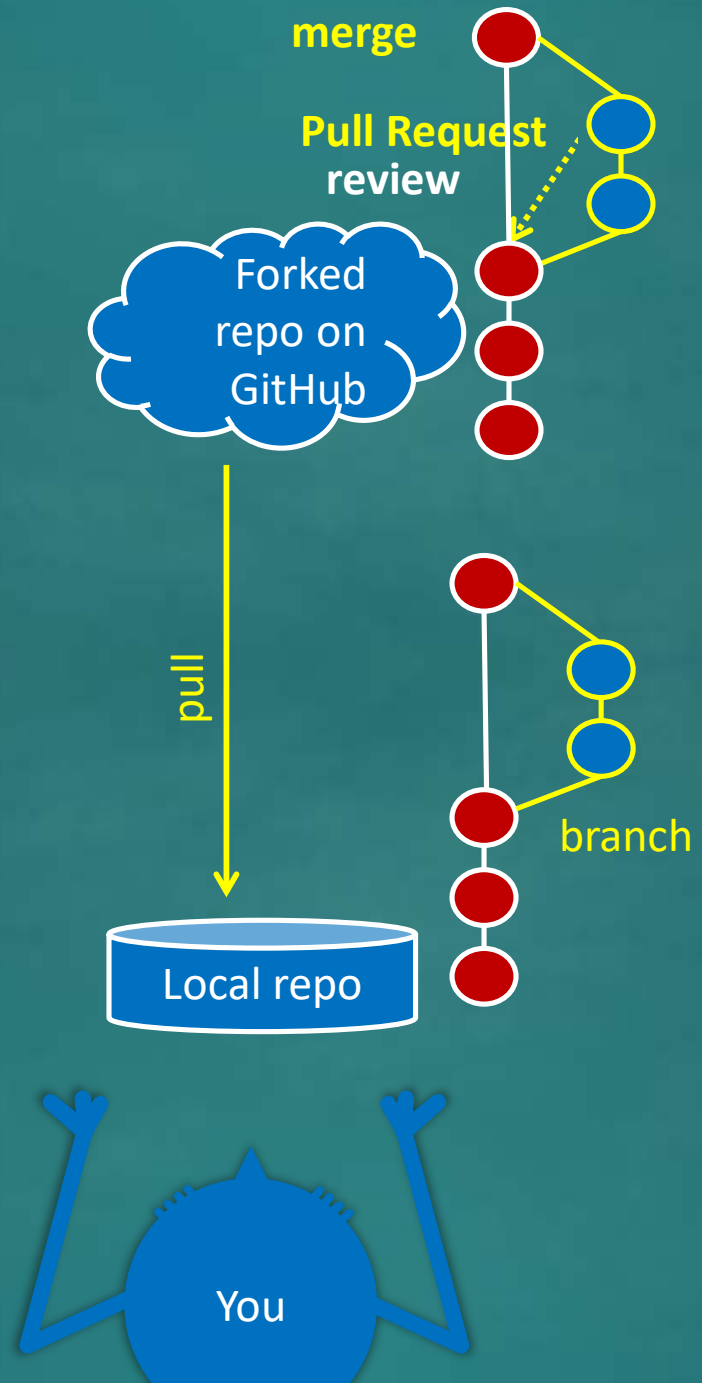
Creating Pull Requests
from **other branches**



You'll be doing something
like this in the tP



Practice managing PRs
using your own iP fork



Week 5

Topics:

- [W5.1] Requirements: Intro ⚡⚡
- [W5.2] Requirements: Gathering ⚡⚡⚡
- [W5.3] Requirements: Specifying ⚡
- [W5.4] Code Quality ⚡
- [W5.5] Refactoring ⚡⚡
- [W5.6] Assertions 🎓
- [W5.7] Java: streams 🎓
- [W5.8] RCS: Managing Pull Requests II ⚡⚡

Admin:

1. Submit weekly quiz P
2. [optional] Submit mid-term f course 🕒 Sat, Feb 17th 2359
3. Prelim peer evaluation on TEAMMATES 🕒 Thu, Feb 15th 2359 P

This feedback will not be used to grade the receiver.

iP:

1. Generate a new JAR file
2. Write some full commit messages
3. Add Increments as PRs: A-Assertions, A-CodeQuality, A-Streams
4. Add Increment: A-CI
5. Add an extension

tP:

1. 👤 Brainstorm user stories 🕒 before the tutorial
2. 👤 Choose user stories for v1.2 🕒 before/during the tutorial

Q

Every team member *must* put in an equal share of work for the tP

☐ True

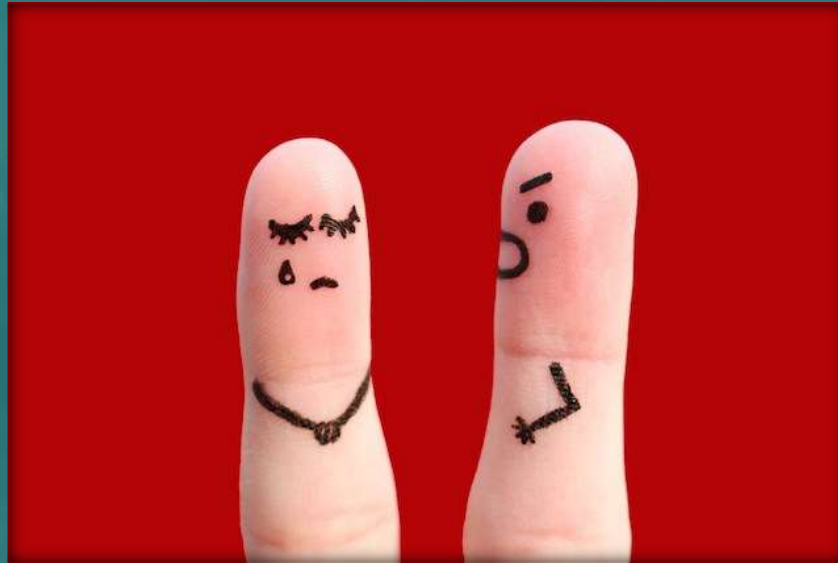
☐ False

The tP grade will vary based on your contribution.
45/50 of tP marks are graded individually.

Promise only what you can deliver,
deliver what you promised,
on time, at required quality.

Stress: how to reduce?

Don't pressure others to do more



Putting less effort for CS2103/T doesn't make someone a bad person.

Cheating, making false promises, being unreliable, being disruptive, etc. ... do.

Have a honest conversation about how each member is bringing to the table (don't ask why).



*Thank
you!*

Dr Ganesh Neelakanta Iyer

gni@nus.edu.sg

<http://ganeshniyer.github.io>

<https://www.linkedin.com/in/ganeshniyer/>